IA-graphie

SAÉ 3.02 — Simulation de trafic distribuée · Groupe Zero Trust

Contributeurs	Lucas Bucher · Benjamin Deiber · Quentin Altinok
Outils déclarés	Gemini (Lucas) · Claude (Benjamin) · ChatGPT (Quentin)

1. Cartographie de l'usage

Les usages sont regroupés par membre et par phase. Les périodes indiquées sont celles qui peuvent être recoupées avec les échanges ou les annexes. La dernière colonne distingue une trace documentaire d'un test réellement prouvé.

Phase / période	Usage de l'IA	Apport et décisions	Trace / preuve
Modèles et simulation Lucas · Gemini *Période non précisée*	Propositions de squelettes pour les véhicules, les feux et le scénario de *simulation.py*.	Code repris et adapté. Les QCM ont ensuite servi à réviser le fonctionnement.	Classes et captures dans l'annexe Lucas.
Socket Quentin · ChatGPT Sept.–oct. 2026	Premières bases des sockets, relecture et correction du client/serveur ; messages JSON délimités par saut de ligne ; lien avec la simulation.	Demandes précisées pour utiliser *sim.step()* et *sim.etat()* plutôt que des données fictives.	*socket_server.py* et *socket_client.py* ; essais locaux rapportés, sans logs joints.
Revue et intégration Benjamin · Claude 18–29 oct. 2026	Diagnostic des signaux PyQt6, du client TCP, des échanges JSON, des imports et des requêtes MariaDB.	Plusieurs correctifs proposés ; conservation du protocole et des signaux retenus par le groupe.	Anomalies décrites dans l'annexe Benjamin, p. 1–2.
Déploiement VM Benjamin · Claude 19–20 oct. 2026	Génération des scripts *setup_vm1_server.sh* et *setup_vm2_client.sh*.	Revue nécessaire : un fichier Python non vide ne prouve pas qu'il s'agit du bon serveur.	Erreur et correction décrites dans l'annexe Benjamin, p. 2.
Fonctions / documents Benjamin · Claude 20–28 oct. 2026	Aide à *info_traffic*, aux conversions GPS/pixels, au README et aux documents de projet.	Propositions sélectionnées, intégrées ou réécrites selon le besoin.	Fichiers du dépôt et annexe Benjamin ; validation détaillée non jointe.
Compréhension Lucas / Quentin *Dates partielles*	Explication de code, simplification des commentaires, QCM et analyse d'erreurs de calcul (Haversine).	Usage pédagogique ; correction des commentaires redondants dans *vehicule_urgence.py*.	Avant/après dans l'annexe Lucas, p. 7–9 ; échanges Quentin.

Périmètre : cette cartographie décrit des usages déclarés, pas un pourcentage calculé de code généré. « Trace » signifie ici fichier, annexe ou résultat rapporté ; aucun log ni numéro de commit n'a été inventé.

2. Exemples de prompts et analyse critique

Trois cas d'usage accompagnés d'une analyse critique. Le prompt de Quentin ci-dessous est une formulation proposée pour cette annexe, et non une citation attestée d'un échange ancien.

01. Quentin / ChatGPT — diagnostic TCP et messages JSON

> **Extrait du prompt du 03/10, abrégé :**
>
> « Je développe la partie réseau d'une simulation de circulation en Python utilisant des sockets TCP et des messages JSON. […] Analyse mes fichiers `socket_server.py` et `socket_client.py` pour identifier les problèmes de connexion, de transmission des messages JSON et de gestion de plusieurs clients. […] Explique les erreurs et propose des pistes de correction sans réécrire mon code. »

Résultat: Une liste de risques liés aux messages TCP/JSON, aux déconnexions et aux connexions simultanées, sans réécriture des fichiers. **Analyse critique :** Point pertinent : TCP transporte un flux ; un recv() ne correspond pas forcément à un JSON complet. Le séparateur de lignes retenu pour le projet répond à ce risque. Le tampon de réception, les déconnexions et l'usage multi-clients restent à contrôler avec des essais réels, sans supposer que l'IA les a validés.

Trace : Fichiers socket_server.py et socket_client.py ; pas de compte rendu de test joint ni de réponse IA associée à cette formulation.

02. Lucas / Gemini — commentaires dans vehicule_urgence.py

> **Extrait du prompt, abrégé :**
> « est-ce que j'ai bien compris le code en le documentant ??? garde ma documentation essaye de la rendre plus facile a comprendre pour les autres […] »

Résultat : Réécriture des commentaires jugés trop nombreux dans le constructeur de vehicule_urgence.py. **Tri critique :** Gardé : l'explication utile de l'héritage de Voiture. Retiré : les commentaires répétant les attributs. Ce choix améliore la lecture, sans prouver à lui seul le bon fonctionnement du code.

Trace : Captures et comparaison avant/après dans l'annexe Lucas, p. 7–9.

03. Benjamin / Claude — revue d'intégration

> **Extrait du prompt du 18/10, abrégé :**
> « Tu es un expert en Python, PyQt6, MariaDB, Sockets TCP et architectures distribuées. Tu vas réaliser une analyse globale d'intégration et de revue de code […] Identifie tous les bugs potentiels […] Fournis les corrections précises des fichiers concernés. »

Résultat : Signalement d'un run() client incorrect, de noms de signaux incompatibles, de connect(self.config) sans ** et de tables MariaDB incohérentes. **Tri critique :** Gardé : les anomalies à corriger. Écarté : la réécriture automatique de tout le protocole JSON ; le signal state_received et les choix d'interface de l'équipe ont été préservés.

Trace : Revue détaillée dans l'annexe Benjamin, p. 1–2 ; aucun identifiant de commit n'y est associé.

3. Réflexion éthique

Maîtrise et dépendance. L'IA a fait gagner du temps, mais un correctif qu'on ne comprend pas devient difficile à maintenir. C'est sensible pour les sockets, les signaux PyQt6 et le déploiement. Notre principe « Zero Trust » s'applique aussi aux réponses de l'IA : comprendre, relire, puis tester. L'erreur de détection du serveur dans un script Bash montre pourquoi.

Sécurité et données. Les scripts proposés peuvent installer des paquets ou modifier la configuration des VM : leur contenu et les droits d'exécution doivent être contrôlés. Le mot de passe MariaDB mentionné dans le projet est un identifiant de test, à ne pas réutiliser en production ni transmettre comme secret réel. La simulation repose sur des véhicules fictifs, sans données personnelles identifiantes ; avec de vraies traces GPS, il faudrait prévoir minimisation, base légale et protection des accès (RGPD).

Biais, sources et méthode. Une IA peut imposer son format JSON, ses noms de signaux ou sa structure de classes. Nous gardons les choix compatibles avec le cahier des charges et les licences des dépendances. Pour la suite, une règle simple est proposée : conserver le prompt utile et sa réponse, noter le fichier modifié, faire relire la modification par un autre membre et joindre un test ou un commit quand il existe.

Sources : contributions de Lucas Bucher , Benjamin Deiber et Quentin Altinok ; échanges de travail sur les fichiers réseau. Les tests sans logs et les dates non établies sont signalés comme tels.